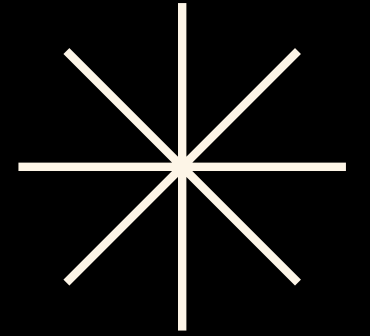
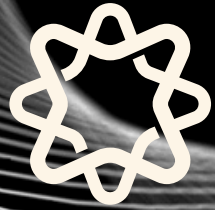
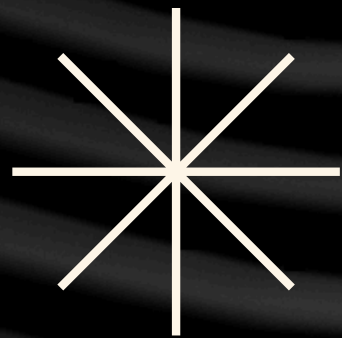


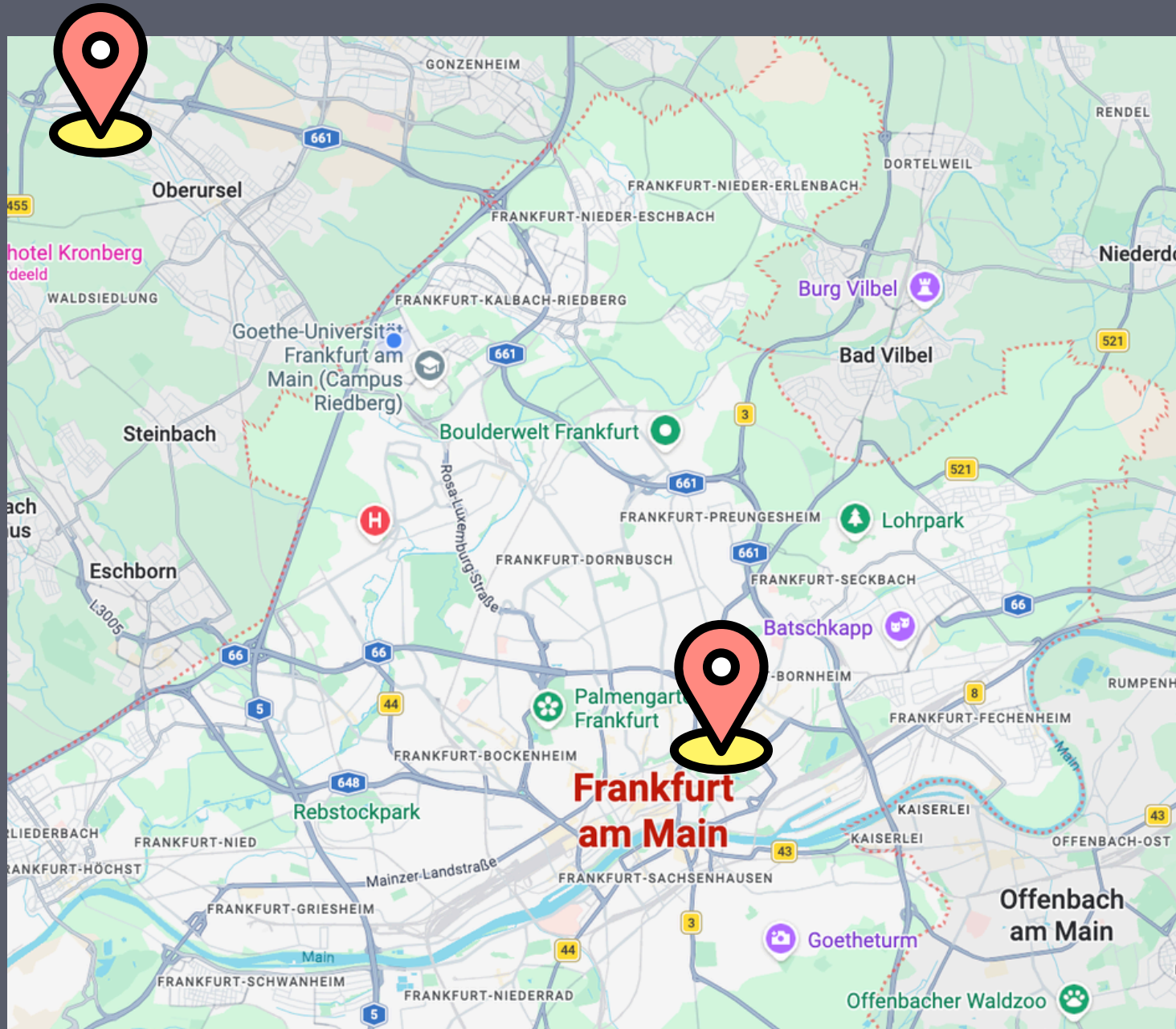


Shortest Path Algorithms

Comparing A, A* & Dijkstra's algorithm



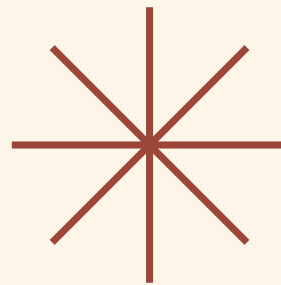
by Zhang Ziyou (Zoe) & Yang Yunle (Kem)



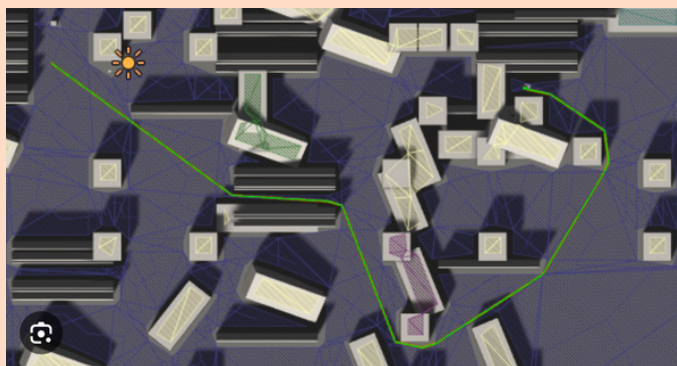
How does Google Maps route planning algorithm work?

Google Maps route planning uses a combination of Dijkstra's algorithm and A* search algorithm to find optimal routes. The system processes real-time traffic data, historical patterns, road conditions, and user preferences to calculate the best path between your starting point and destination. The algorithm considers factors like traffic speed, turn difficulty, road restrictions, and predicted conditions during your journey time.

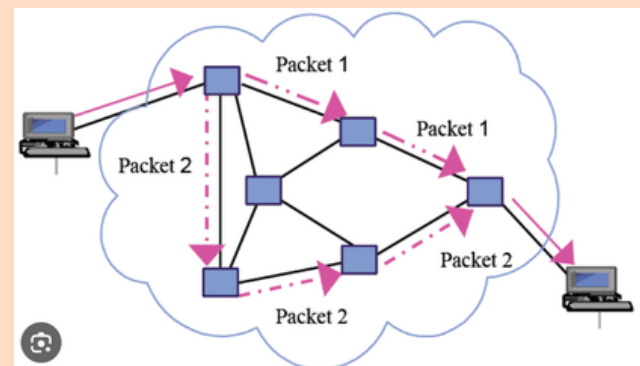
Applications



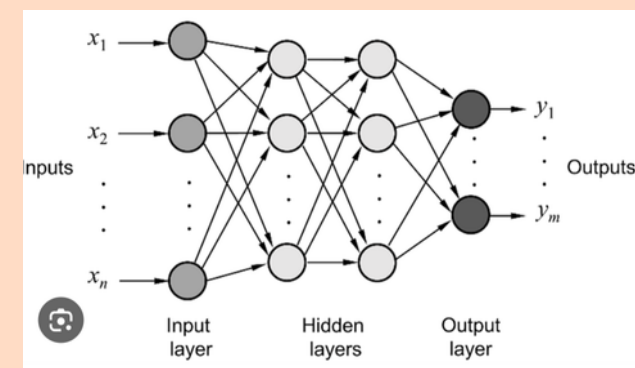
Video Games & NPC Pathfinding



Internet & Network Routing



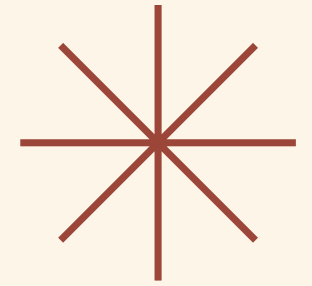
AI Planning & Decision Systems



GPS & Navigation Systems



Table of Content



Introduction

- Research Question
- Hypothesis

Data & Analysis

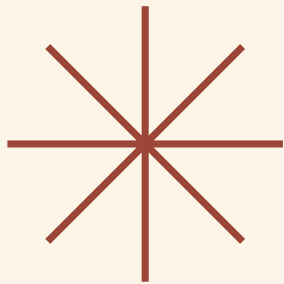
- Execution time, path length, etc. of each algorithm (graphs)

Methodology

- Map type & general rules
- Variables

Conclusion & Evaluation

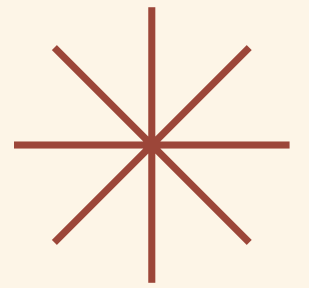
- Determine "best" algorithm
-





Introduction

Introduction



01

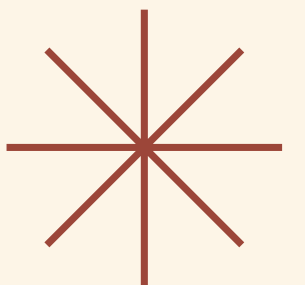
Research Question

What is the trade-off between accuracy and runtime in Dijkstra's algorithm, A, and A* when applied to grid-based pathfinding problems?

02

Hypothesis

- A $\Rightarrow$ fastest, but may not be optimal
- Dijkstra $\Rightarrow$ slowest, but highest accuracy
- A* $\Rightarrow$ often faster than Dijkstra, finds shortest path





Methodology

Method explanation

01

60 grid based maps

- 512 x 512 pixel map
- Movement: 8 directional
- Obstacles (6 different densities)
- Start: first upper left square available
- End: first lower right square available

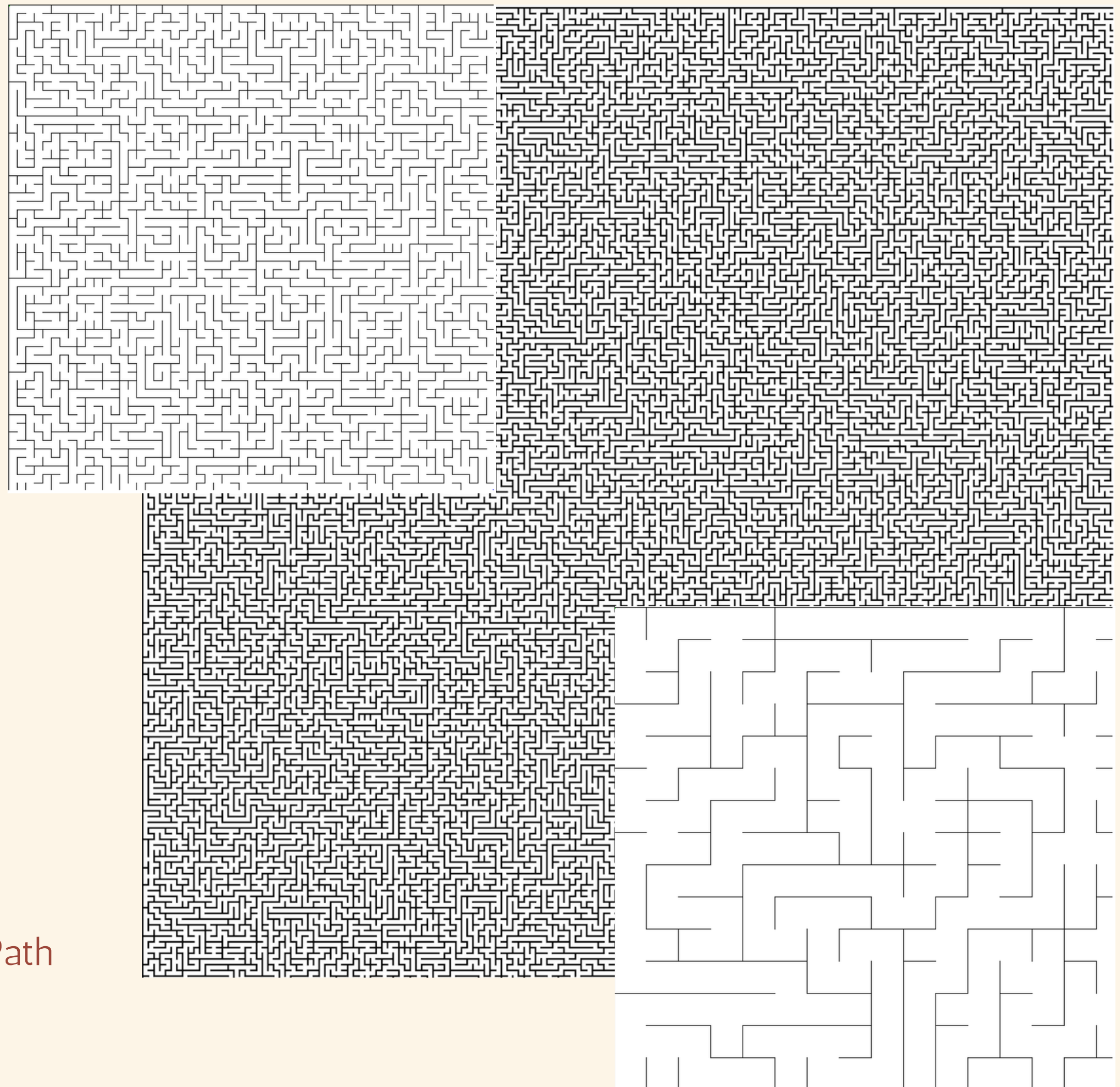
02

Independent Variable:

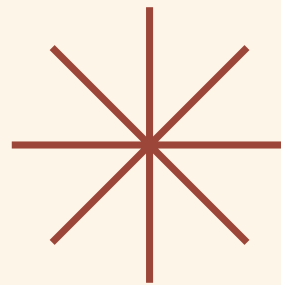
Algorithms: Dijkstra's, A, and A*

Dependent Variable:

Execution Time, Memory usage,
Nodes/tiles explored, Path Length, Path
cost



Algorithms



A Algorithm

- "Greedy" → tries to head straight for finish
- Goes to tile with least distance to finish

A* Algorithm

- adds distance to start & finish
- goes to tile with lowest sum

Dijkstra

- Goes to tile with least distance from start
- covers all squares until finish

```
<int, int>> astar(const GridMap &m,
                  pair<int, int> start,
                  pair<int, int> goal,
                  int &nodesExpanded)

nodesExpanded = 0;

if (!inBounds(start.first, start.second) ||
    !inBounds(goal.first, goal.second))

    return << "Start or goal out of bounds.\n";
    return {};

if (!isWalkable(start.first, start.second) ||
    !isWalkable(goal.first, goal.second))

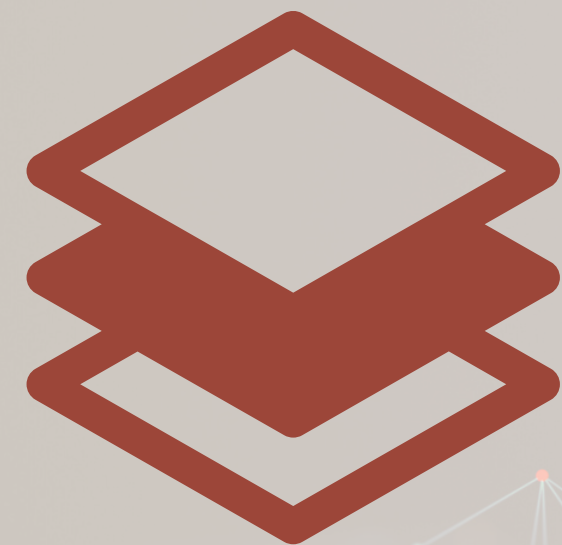
    return << "Start or goal not walkable.\n";
    return {};

double INF = 1e30;
vector<double>> gScore(m.height, vector<double>(m.width));
vector<bool>> closed(m.height, vector<bool>(m.width, false));
vector<pair<int, int>>> parent(m.height, vector<pair<int, int>>(m.width, {-1, -1}));

priority_queue<State, vector<State>, CompareState> open;

gScore[start.second][start.first] = 0.0;
h0 = octileHeuristic(start.first, start.second, goal.first, goal.second);
push({start.first, start.second, 0.0, h0});

// Directions: (dx,dy)
int DX[8] = {1, -1, 0, 0, 1, 1, -1, -1};
int DY[8] = {0, 0, 1, -1, 1, -1, 1, -1};
double COST[8] = {
    1.0, 1.0, 1.0, 1.0, // cardinal
    sqrt(2.0), sqrt(2.0),
    sqrt(2.0), sqrt(2.0)};
```

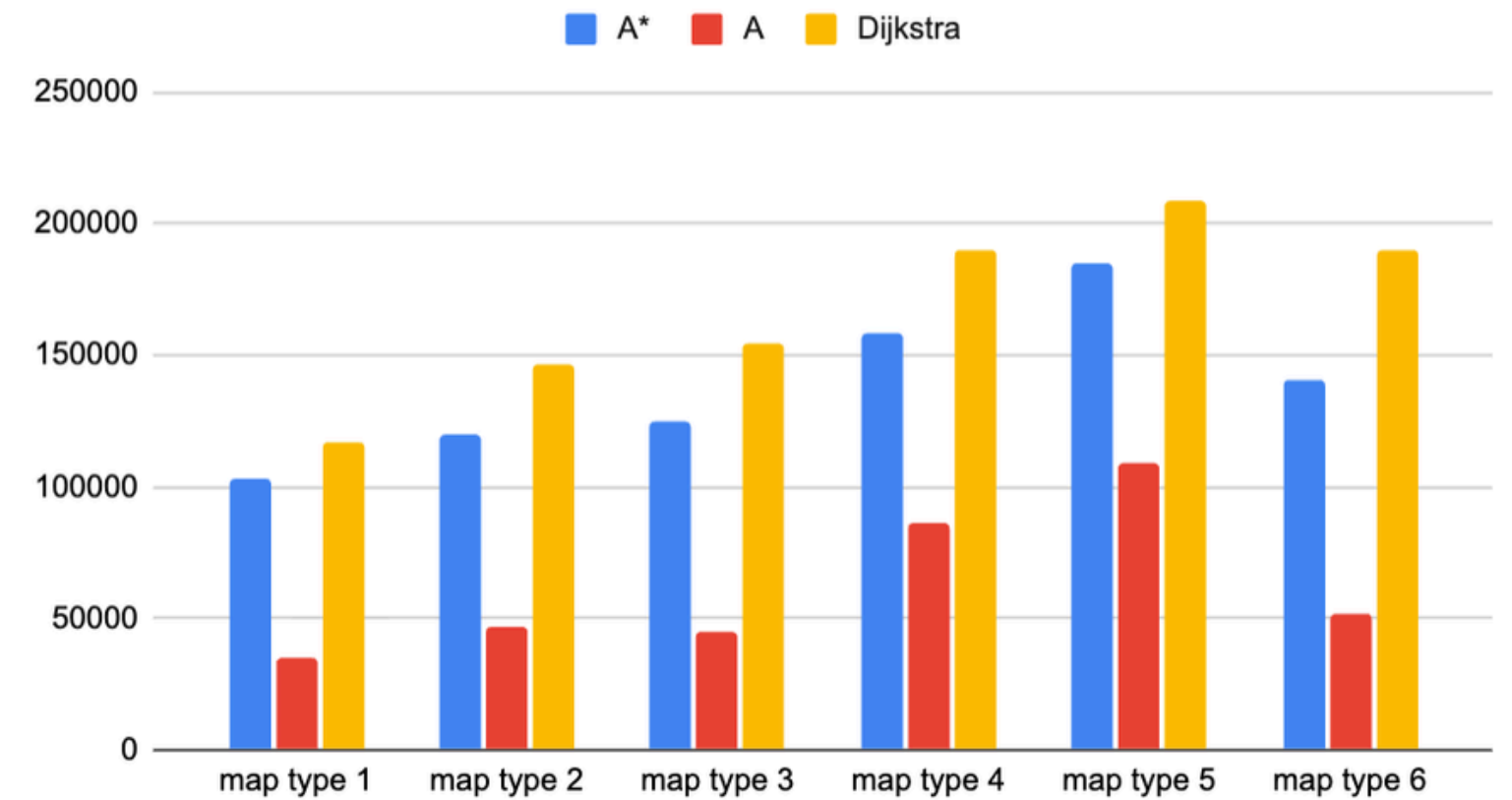


Data & Analysis

Nodes explored

- A fewest nodes explored
- A* a bit less than Dijkstra
 - Considering distance to goal
→ narrows search
- Dijkstra most
 - Expands uniformly outward to cover whole map

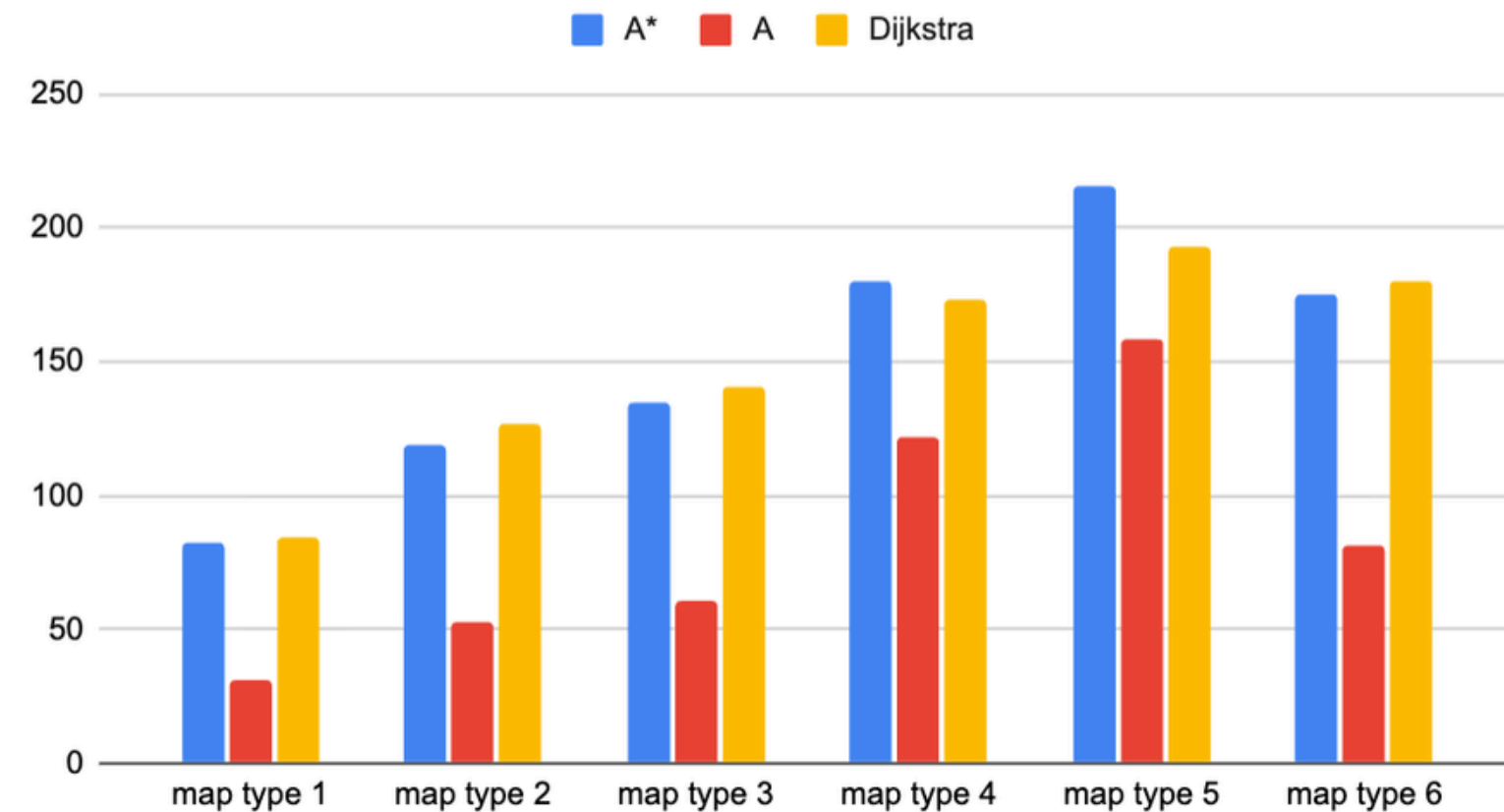
Nodes Explored for each Algorithm



Execution Time

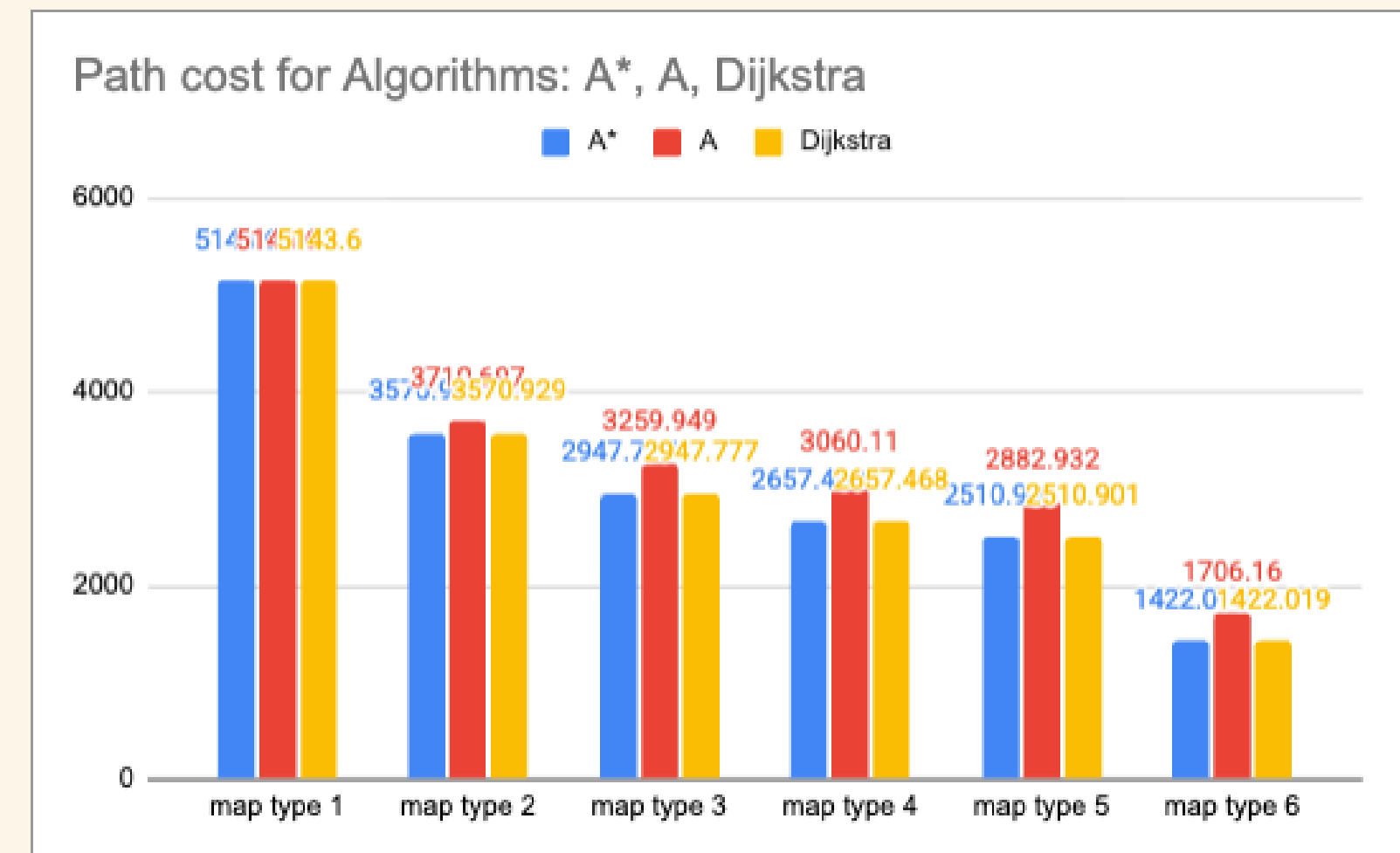
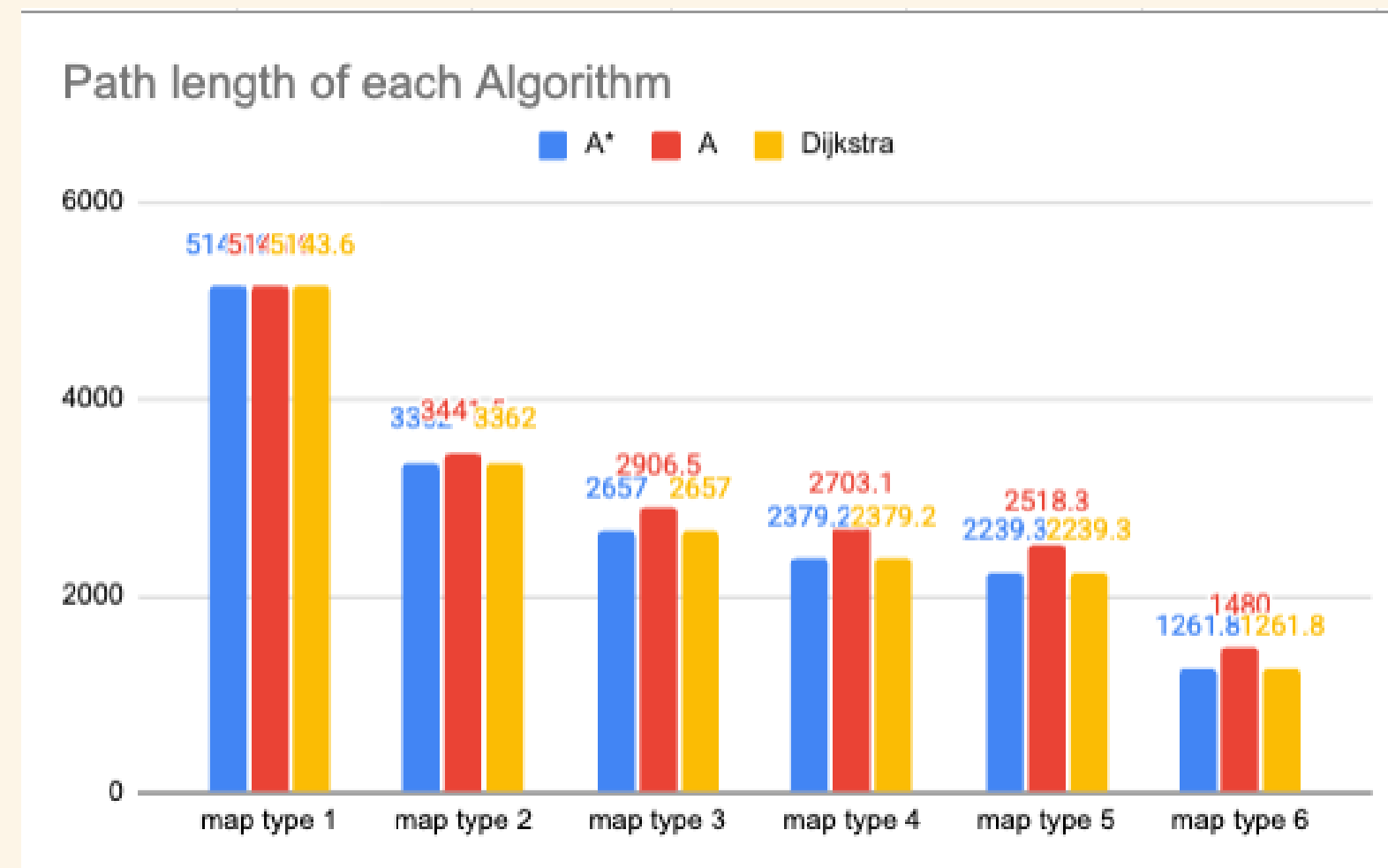
- A algorithm fastest
- A* slow
- Dijkstra slowest (kinda...)

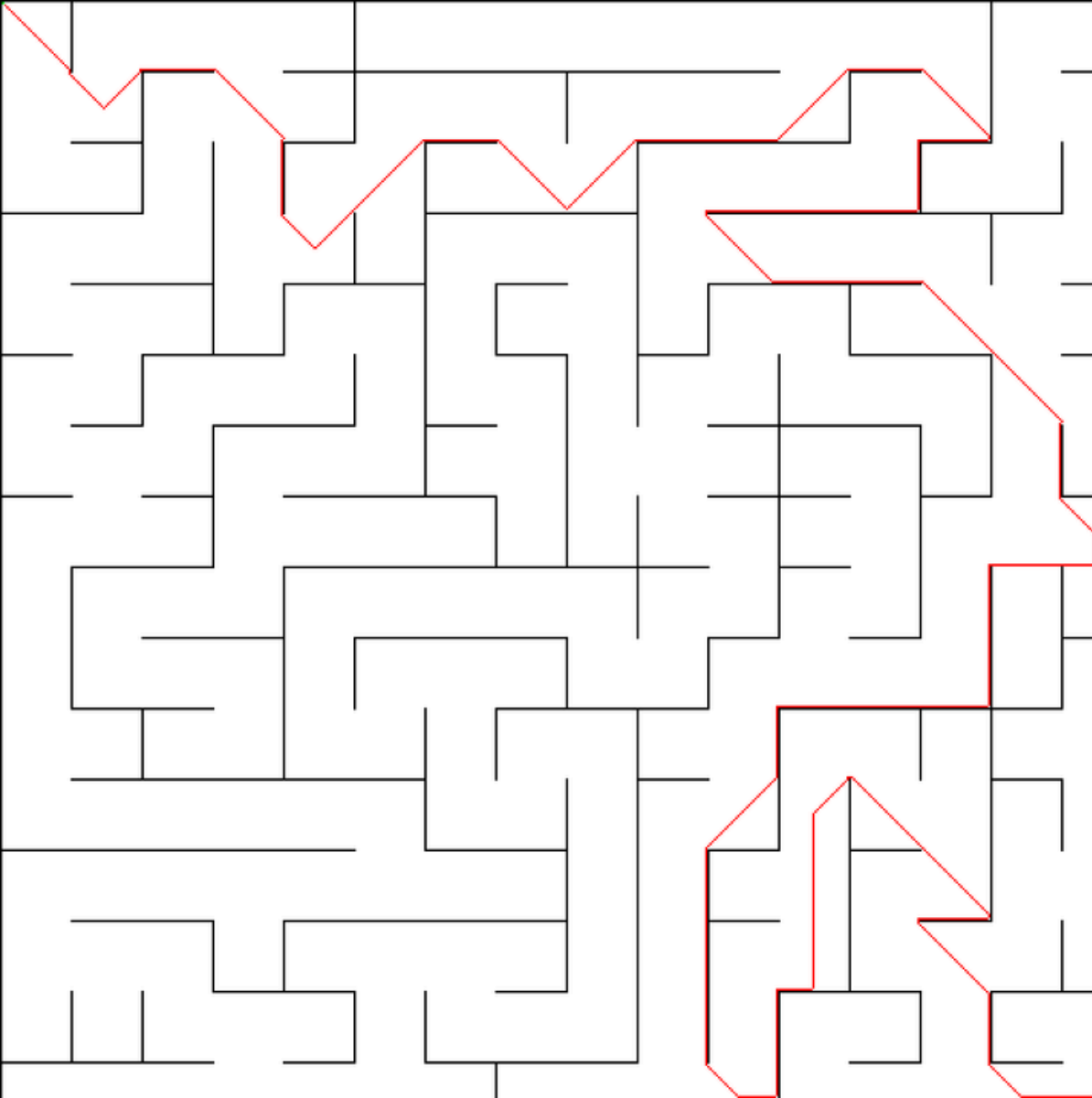
Execution Time of each Algorithm



Path length & Path cost

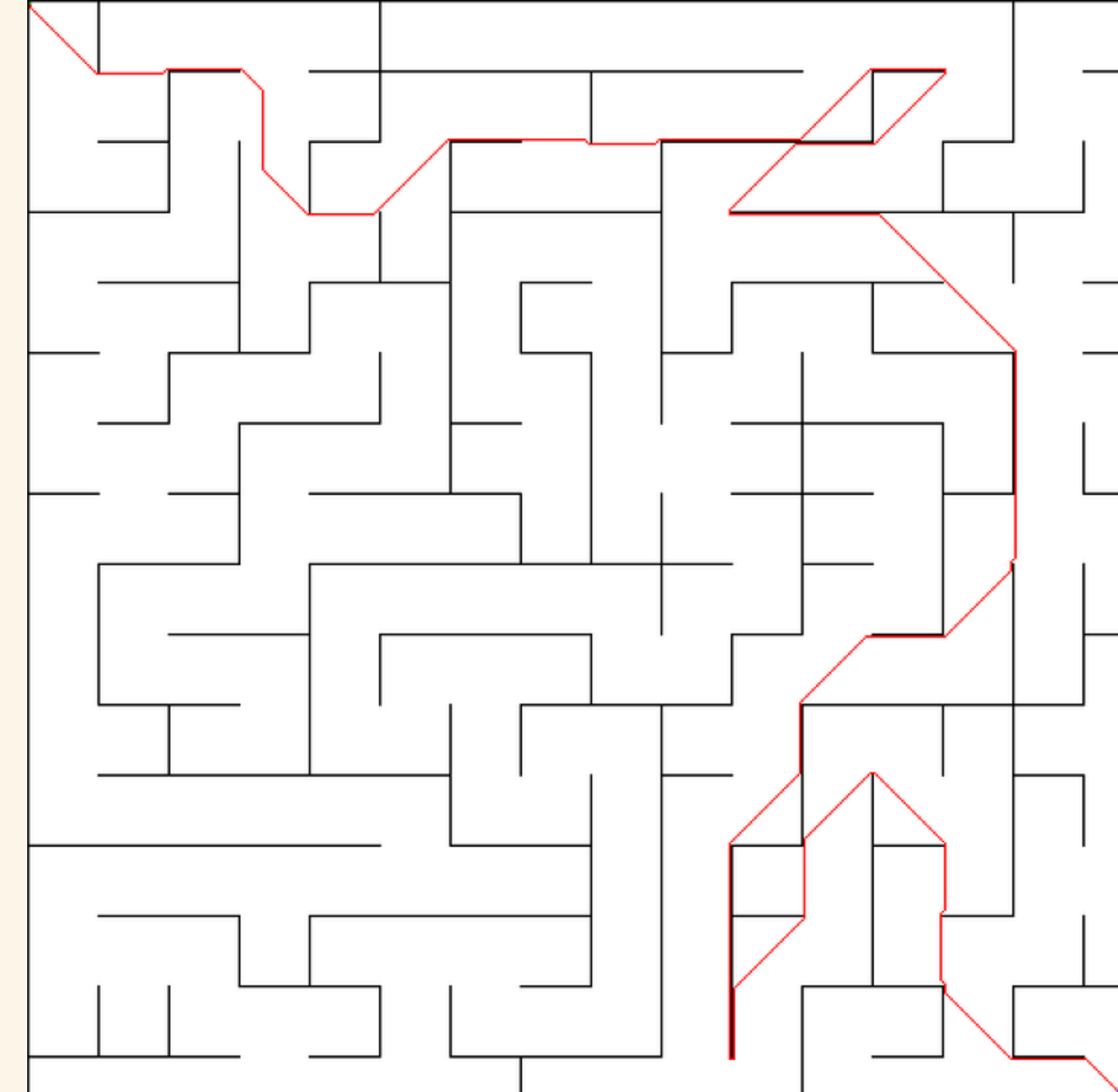
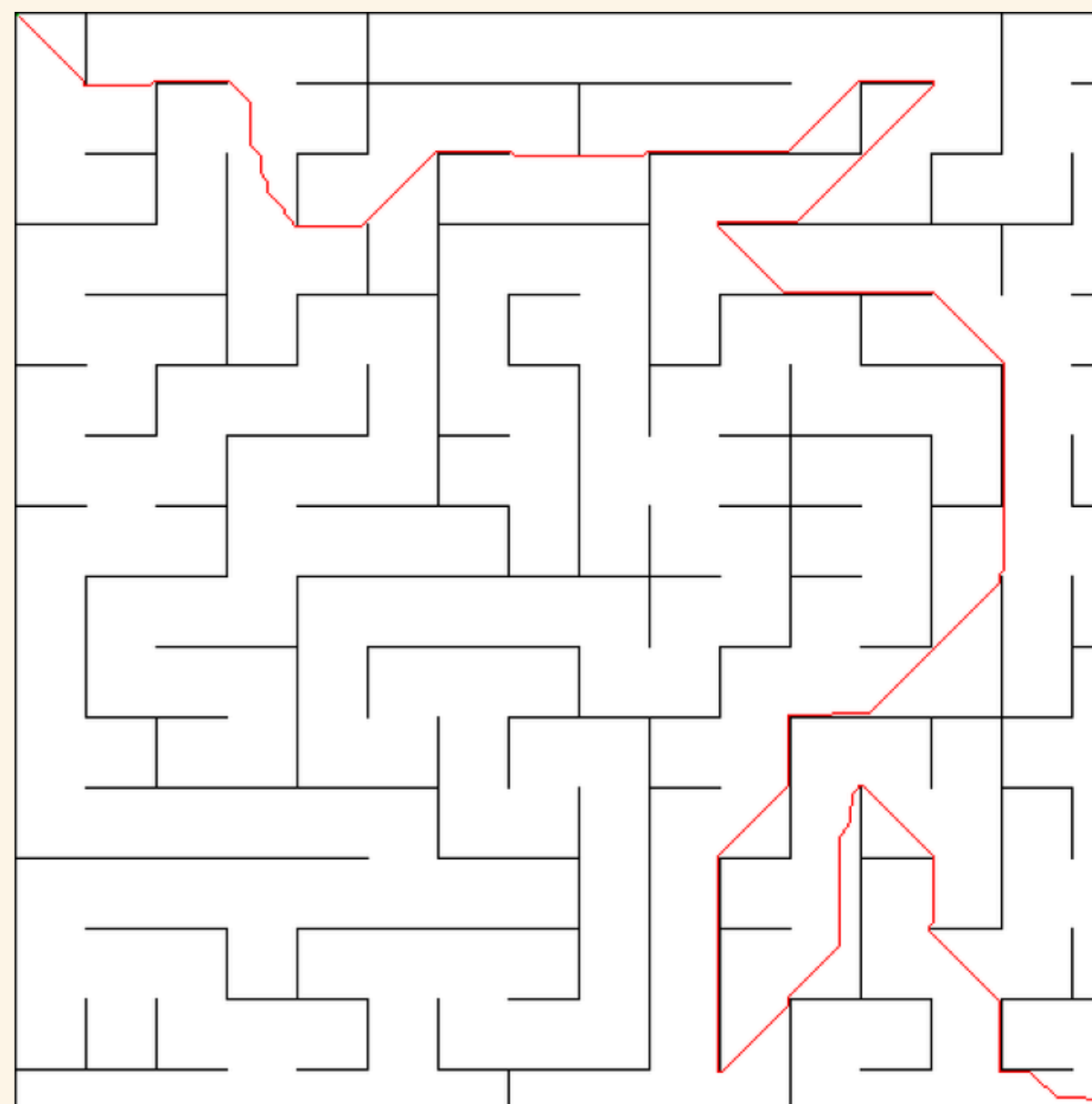
- Map 1:
 - Identical values - outlier
- Map 2 - 6:
 - Dijkstra and A* identical → optimal
 - A nearly always longer path
 - More freedom, more chances to go wrong



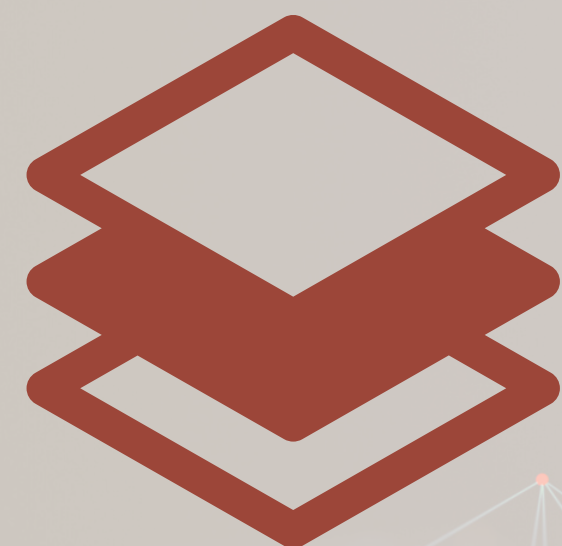


A Algorithm

A* Algorithm



Dijkstra



Conclusion & Evaluation

Conclusion

Clear trade-off between accuracy and runtime

- **Dijkstra**
 - Highest node expansion
 - Longest runtime
 - Consistently optimal path cost
 - No heuristic guidance → exhaustive exploration
- **A***
 - Significantly fewer nodes explored than Dijkstra
 - Lower runtime
 - Maintains optimal path cost
 - Most balanced algorithm overall
- **A**
 - Fewest nodes explored
 - Lowest runtime
 - Produces suboptimal paths

Evaluation

Strengths of Investigation

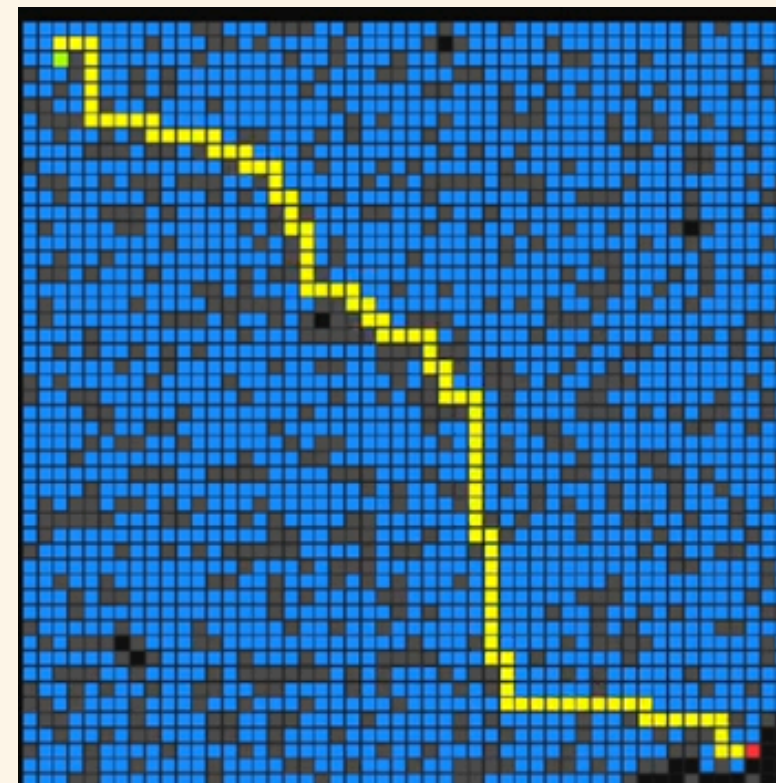
- Multiple map types tested
- Direct comparison under identical conditions
- Both runtime and path quality measured
- Node expansion included → strengthens explanation of runtime differences

Improvements & Further Research

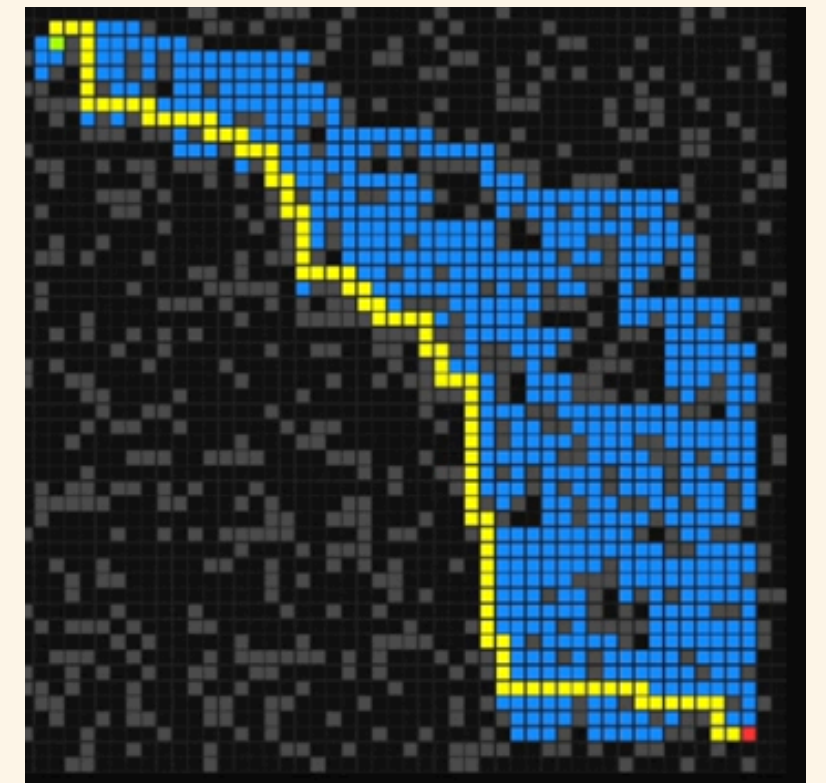
- Weighted A* → Test multiple heuristic functions
- Increase number of randomized map trials
- Analyze scalability (larger grids: $200 \times 200 +$)
- Test real-time replanning performance
- Compare with additional algorithms (e.g., D* Lite, BFS for baseline)

Limitations

- Grid-based environments only
- Does not represent continuous space pathfinding
- No dynamic or real-time scenarios
- Hardware/runtime environment not deeply controlled
→ Runtime can vary due to system performance
- A* not Weighted A*



Dijkstra



Weighted A*

Works Cited

AI model response to prompt about A* code and requests for “A” and “Dijkstra” algorithms.

Anthropic Claude. Personal communication, 2026.

“Dijkstra vs A* – What's the Difference?” YouTube, uploaded by freeCodeCamp.org, 16 Apr.

2021, www.youtube.com/watch?v=EW9FeQtMv7U.

“Dijkstra’s Shortest Path Algorithm Explained | With Example.” YouTube, uploaded by

MyCodeSchool, 3 July 2016, www.youtube.com/watch?v=bZkzH5x0SKU.

“A* Search Algorithm.” Wikipedia: The Free Encyclopedia, Wikimedia Foundation, 6 Feb.

2026, en.wikipedia.org/wiki/A*_search_algorithm. Accessed 12 Feb. 2026.

“Dijkstra’s Algorithm Visualization and Explanation.” YouTube, uploaded by Computerphile, 22

Aug. 2013, www.youtube.com/watch?v=HMAHrQHmrUQ.

“Dijkstra’s Algorithm.” Wikipedia: The Free Encyclopedia, Wikimedia Foundation, 10 Feb.

2026, en.wikipedia.org/wiki/Dijkstra%27s_algorithm. Accessed 12 Feb. 2026.

“Pathfinding Visualization.” YouTube, www.youtube.com/watch?v=i0x5fj4PqP4.

“Dijkstra vs A* (Short Explanation).” YouTube Shorts,

www.youtube.com/shorts/aXdS8QC2f1M.

“Graph Search Algorithms Explained.” YouTube, www.youtube.com/watch?v=9REexHx0hDY.



I hope you learn something
new today!